

UNIVERSIDAD PERUANA LOS ANDES

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas y Computación



UPLA



PORTAFOLIO ACADÉMICO - SEMANA 14

Tema: AUTOMATIZACIÓN Y MANTENIMIENTO



Curso: Base de Datos II



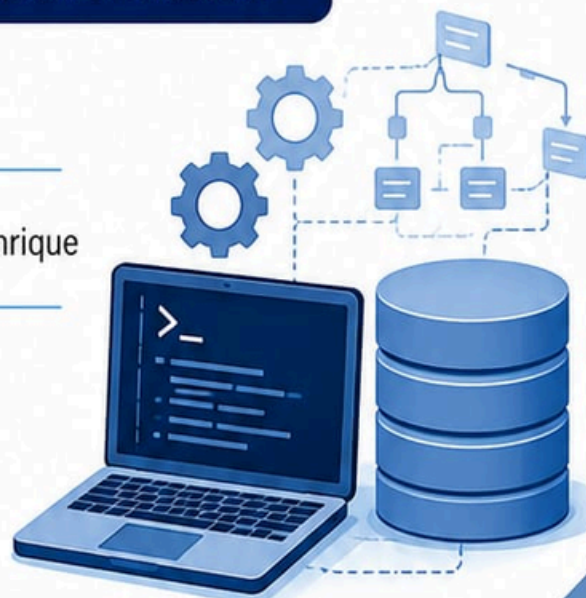
Docente: Ing. Fernandez Bejarano Raul Enrique



Estudiante: Diego Vilchez Estrada



Código / Ciclo: V Ciclo



Huancayo - Perú

2026

TEMA 1: Creación de Jobs y Alerts con SQL Server Agent

1



REPORTE DIARIO DE TRÁMITES A LAS 08:00 AM Y GUARDADO EN TABLA AuditoriaJobs

ENUNCIADO

Generar un reporte diario de los trámites registrados el día anterior a las 08:00 AM y guardarlo en la tabla AuditoriaJobs.

JUSTIFICACIÓN TÉCNICA

Permite tener un historial diario de la actividad de trámites para auditoría y control operativo.

BUENAS PRÁCTICAS

- Se utiliza EXECUTE AS OWNER para seguridad.
- Se programa en horarios de baja carga.
- Se registra fecha/hora de ejecución.

SCRIPT (T-SQL)

```
USE msdb;
GO
-- 1. Crear el Job
EXEC sp_add_job
    @job_name = N'Reporte Diario de Trámites',
    @enabled = 1,
    @description = N'Reporte diario de trámites a las 08:00 AM y guardado en AuditoriaJobs';

GO
-- 2. Crear el Step del Job
EXEC sp_add_jobstep
    @job_name = N'Reporte Diario de Trámites',
    @step_name = N'Generar Reporte',
    @subsystem = N'TSQL',
    @command = N'
        INSERT INTO dbo.AuditoriaJobs (NombreJob, FechaEjecucion, Detalle,
        RegistrosAfectados)
        SELECT
            N''Reporte Diario de Trámites'',
            GETDATE(),
            N''Trámites del día anterior'',
            COUNT(*)
        FROM dbo.Trámite
        WHERE CAST(FechaRegistro AS DATE) = CAST(DATEADD(day,-1,GETDATE()) AS DATE);',
    @database_name = N'GradosTitulos',
    @on_success_action = 1; -- Salir

GO
-- 3. Programar el Job a las 08:00 AM todos los días
EXEC sp_add_schedule
    @schedule_name = N'Diario 08:00 AM',
    @freq_type = 1, -- Diario
    @freq_interval = 1,
    @active_start_time = @@@0000; -- 08:00:00 AM
EXEC sp_attach_schedule
    @job_name = N'Reporte Diario de Trámites',
    @schedule_name = N'Diario 08:00 AM';
EXEC sp_add_jobserver @job_name = N'Reporte Diario de Trámites';
```

CONSOLA DE SALIDA

```
[2025-06-20 07:00:00] Iniciando Job
'Reporte Diario de Trámites'.

[2025-06-20 08:00:01] Ejecutando paso
'Generar Reporte'.

[2025-06-20 08:00:02] Conectando a la base
de datos 'GradosTitulos'.

[2025-06-20 08:00:03] Insertando datos en
AuditoriaJobs...

[2025-06-20 08:00:04] Registros
insertados: 142

[2025-06-20 08:00:05] Paso completado
exitosamente.

[2025-06-20 08:00:06] Job completado
exitosamente.
```

SUCCESS 

2



AUTOMATIZACIÓN SEMANAL DE ACTUALIZACIÓN DE FechaUltMod PARA TRÁMITES 'EN PROCESO'

ENUNCIADO

Actualizar semanalmente el campo FechaUltMod para los trámites que se encuentren con estado 'EN PROCESO'.

JUSTIFICACIÓN TÉCNICA

Garantiza que la fecha de última modificación refleje actividad reciente, útil para reportes y seguimiento.

BUENAS PRÁCTICAS

- Se actualizan solo registros necesarios.
- Se ejecuta semanalmente en horario off-peak.
- Se usa GETDATE() para consistencia.

SCRIPT (T-SQL)

```
USE msdb;
GO
EXEC sp_add_job
    @job_name = N'Actualizar FechaUltMod Trámites',
    @enabled = 1,
    @description = N'Actualización semanal de FechaUltMod para
    trámites EN PROCESO';

GO
EXEC sp_add_jobstep
    @job_name = N'Actualizar FechaUltMod Trámites',
    @step_name = N'Actualizar FechaUltMod',
    @subsystem = N'TSQL',
    @command = N'
        UPDATE dbo.Trámite
        SET FechaUltMod = GETDATE()
        WHERE Estado = N''EN PROCESO'';
        -- Se actualiza solo los registros necesarios
        WHERE Estado = N''GradosTitulos'';';
    @database_name = N'GradosTitulos',
    @on_success_action = 1;

GO
EXEC sp_add_schedule N'Semanal - Domingo 02:00 AM',
    @freq_type = 1, -- Semanal
    @freq_interval = 1, -- Domingo
    @active_start_time = @@@0000; -- 02:00:00 AM

GO
EXEC sp_attach_schedule
    @job_name = N'Actualizar FechaUltMod Trámites',
    @schedule_name = N'Semanal - Domingo 02:00 AM';
EXEC sp_add_jobserver @job_name = N'Actualizar FechaUltMod Trámites';
```

CONSOLA DE SALIDA

```
[2025-06-22 02:00:00] Iniciando Job
'Actualizar FechaUltMod Trámites'.

[2025-06-22 02:00:01] Ejecutando paso
'Actualizar FechaUltMod'.

[2025-06-22 02:00:02] Conectando a la base
de datos 'GradosTitulos'.

[2025-06-22 02:00:03] Actualizando registros
con estado 'EN PROCESO'...

[2025-06-22 02:00:04] Registros actualizados:
87

[2025-06-22 02:00:05] Paso completado
exitosamente.

[2025-06-22 02:00:06] Job completado
exitosamente.
```

SUCCESS 

3



PURGA MENSUAL DE REGISTROS DE TramiteLogTemporal MAYORES A 180 DÍAS HACIA TABLA HISTÓRICA

ENUNCIADO

Mover mensualmente los registros de TramiteLogTemporal mayores a 180 días a la tabla TramiteLogHistorico y luego eliminarlos de la tabla temporal, usando transacciones y manejo de errores con TRY...CATCH.

JUSTIFICACIÓN TÉCNICA

Evita crecimiento innecesario de la tabla temporal y conserva el histórico en otra tabla.

BUENAS PRÁCTICAS

- Uso de transacciones para integridad.
- Manejo de errores con TRY...CATCH.
- Registro de actividad en AuditoriaJobs.

SCRIPT (T-SQL)

```
USE msdb;
GO
EXEC sp_add_job
    @job_name = N'Purga Mensual TramiteLogTemporal',
    @enabled = 1,
    @description = N'Mover registros > 180 días a histórico y los elimina';

GO
EXEC sp_add_jobstep
    @job_name = N'Purga Mensual TramiteLogTemporal',
    @step_name = N'Purga Mensual TramiteLogTemporal',
    @subsystem = N'TSQL',
    @command = N'
        BEGIN TRY
            -- Mover registros a historico
            INSERT INTO dbo.TramiteLogHistorico
            SELECT * FROM dbo.TramiteLogTemporal
            WHERE FechaLog < DATEADD(day,-180,GETDATE());

            -- Eliminar registros de la tabla temporal
            DELETE FROM dbo.TramiteLogTemporal
            WHERE FechaLog < DATEADD(day,-180,GETDATE());

            -- Registrar actividad en AuditoriaJobs
            INSERT INTO dbo.AuditoriaJobs (NombreJob, FechaEjecucion, Detalle, RegistrosAfectados)
            VALUES (N''Purga Mensual TramiteLogTemporal'', GETDATE(), N''Registros > 180 días'', @@ROWCOUNT);

        COMMIT TRAN;
    END TRY
    BEGIN CATCH
        -- ROLLBACK TRAN;
        INSERT INTO dbo.AuditoriaJobs (NombreJob, FechaEjecucion, Detalle, RegistrosAfectados)
        VALUES (N''Purga Mensual TramiteLogTemporal'', GETDATE(), N''Error en la purga'', @@ROWCOUNT);
    END CATCH;

    -- Se actualiza solo los registros necesarios
    WHERE Estado = N''GradosTitulos'';';
    @database_name = N'GradosTitulos',
    @on_success_action = 1;

GO
EXEC sp_add_schedule N'Mensual - Día 1 02:00 AM',
    @freq_type = 1, -- Mensual
    @freq_interval = 1, -- Día 1
    @active_start_time = @@@0000; -- 02:00:00 AM

GO
EXEC sp_attach_schedule
    @job_name = N'Purga Mensual TramiteLogTemporal',
    @schedule_name = N'Mensual - Día 1 02:00 AM';
EXEC sp_add_jobserver @job_name = N'Purga Mensual TramiteLogTemporal';
```

CONSOLA DE SALIDA

```
[2025-06-01 01:00:00] Iniciando Job
'Purga Mensual TramiteLogTemporal'.

[2025-06-01 01:00:01] Ejecutando paso
'Purga y Movimiento'.

[2025-06-01 01:00:02] Iniciando transacción...

[2025-06-01 01:00:03] Moviendo registros >
180 días a TramiteLogHistorico...

[2025-06-01 01:00:04] Registros movidos: 5,423

[2025-06-01 01:00:05] Eliminando registros
de TramiteLogTemporal...

[2025-06-01 01:00:06] Registros eliminados: 5,423

[2025-06-01 01:00:07] Transacción confirmada.

[2025-06-01 01:00:08] Paso completado
exitosamente.

[2025-06-01 01:00:09] Job completado
exitosamente.
```

SUCCESS 



CREACIÓN DE UN ALERT EN SQL SERVER AGENT PARA ERRORES DE SEVERIDAD >= 17 CON OPERADOR NOTIFICADO

ENUNCIADO

Crear un Alert en SQL Server Agent para capturar errores de severidad mayor o igual a 17 y notificar a un Operador configurado.

JUSTIFICACIÓN TÉCNICA

Permite reaccionar rápidamente ante errores críticos que pueden afectar la disponibilidad del servicio.

BUENAS PRÁCTICAS

- Alertas para severidad crítica (>=17).
- Notificación inmediata vía Database Mail.
- Operador definido con contacto válido.

SCRIPT (T-SQL)

```
USE msdb;
GO
-- 1. Crear Operador
EXEC sp_add_operator
    @name = N'DBA_Operador',
    @enabled = 1,
    @email_address = N'dba@upla.edu.pe',
    @notification_enabled = 1;

GO
-- 2. Crear Alert
EXEC sp_add_alert
    @name = N'Error Severidad >= 17',
    @message_id = 0,
    @severity = 17,
    @delay_between_responses = 1, -- minutos
    @include_event_description_in = 1;

GO
-- 3. Asignar Alert con Operador
EXEC sp_add_notification
    @alert_name = N'Error Severidad >= 17',
    @operator_name = N'DBA_Operador',
    @notification_method = 1; -- Email
```

CONSOLA DE SALIDA

```
[2025-06-18 14:23:10] ALERT TRIGGERED
Error Severidad >= 17

[2025-06-18 14:23:10] Severidad: 20 )
[2025-06-18 14:23:10] Mensaje: El servicio
SQL Server no puede continuar debido a
un error crítico.

[2025-06-18 14:23:10] Enviando notificación
a Operador 'DBA_Operador'...

[2025-06-18 14:23:11] Email enviado a
dba@upla.edu.pe

[2025-06-18 14:23:11] Alerta procesada
exitosamente.
```

SUCCESS 

5



SISTEMA AUTOMÁTICO DIARIO DE TRÁMITES PENDIENTES > 30 DÍAS CON ENVÍO DE REPORTE VÍA DATABASE MAIL

ENUNCIADO

Generar diariamente un reporte de trámites pendientes con más de 30 días y enviarlo por correo electrónico mediante Database Mail.

JUSTIFICACIÓN TÉCNICA

Ayuda a la gestión proactiva de trámites atrasados y mejora el seguimiento.

BUENAS PRÁCTICAS

- Uso de Database Mail para envío automático.
- Reporte en formato HTML legible.
- Programación diaria en horario adecuado.

SCRIPT (T-SQL)

```
USE msdb;
GO
-- 1. Crear el Job
EXEC sp_add_job
    @job_name = N'Reporte Diario Trámites Pendientes > 30 Días',
    @description = N'Envío reporte diario de trámites pendientes > 30 días';

GO
EXEC sp_add_jobstep
    @job_name = N'Reporte Diario Trámites Pendientes > 30 Días',
    @step_name = N'Generar y Enviar Reporte',
    @subsystem = N'TSQL',
    @command = N'
        -- Generar reporte de trámites pendientes > 30 días
        SELECT * FROM dbo.Trámite
        WHERE FechaRegistro < DATEADD(day,-30,GETDATE());

        -- Enviar reporte por correo
        EXEC msdb.dbo.sp_send_dbmail
            @recipients = N'dba@upla.edu.pe',
            @subject = N''Reporte Diario Trámites Pendientes > 30 Días'',
            @body = N''Reporte de trámites pendientes > 30 días'',
            @body_format = N'HTML',
            @attach_images = 0;

        -- Registrar actividad en AuditoriaJobs
        INSERT INTO dbo.AuditoriaJobs (NombreJob, FechaEjecucion, Detalle, RegistrosAfectados)
        SELECT
            N''Reporte Diario Trámites Pendientes > 30 Días'',
            GETDATE(),
            N''Trámites pendientes > 30 días'',
            COUNT(*)
        FROM dbo.Trámite
        WHERE FechaRegistro < DATEADD(day,-30,GETDATE());';
    @database_name = N'GradosTitulos',
    @on_success_action = 1;

GO
EXEC sp_add_schedule
    @schedule_name = N'Diario 07:30 AM',
    @freq_type = 1, -- Diario
    @freq_interval = 1,
    @active_start_time = @@@0000; -- 07:30:00 AM

GO
EXEC sp_attach_schedule
    @job_name = N'Reporte Diario Trámites Pendientes > 30 Días',
    @schedule_name = N'Diario 07:30 AM';
```

CONSOLA DE SALIDA

```
[2025-06-20 07:30:00] Iniciando Job
'Reporte Diario Trámites Pendientes > 30 Días'.

[2025-06-20 07:30:01] Ejecutando paso
'Generar y Enviar Reporte'.

[2025-06-20 07:30:02] Generando reporte de
trámites pendientes > 30 días...

[2025-06-20 07:30:03] Trámites encontrados: 63

[2025-06-20 07:30:04] Enviando correo vía
Database Mail...

[2025-06-20 07:30:05] Correo enviado a
gestion@upla.edu.pe

[2025-06-20 07:30:06] Paso completado
exitosamente.

[2025-06-20 07:30:06] Job completado
exitosamente.
```

SUCCESS 

TEMA 2: Uso de Database Maintenance Plans

1 BACKUP DIARIO COMPLETO DE 'GRADOS TITULOS' CON COMPRESIÓN Y VERIFICACIÓN HABILITADA

ENUNCIADO

Realizar un backup diario completo de la base de datos 'GradosTitulos' utilizando compresión y verificación habilitada para garantizar integridad y optimizar espacio de almacenamiento.

JUSTIFICACIÓN

Asegura la disponibilidad y recuperación de la información ante fallos, minimizando el espacio ocupado en disco y validando la integridad del backup.

BUENAS PRÁCTICAS

- ✓ Usar compresión para optimizar espacio.
- ✓ Verificación habilitada (CHECKSUM).
- ✓ Ruta dedicada y rotación de backups.
- ✓ Monitoreo de ejecución y alertas.

SCRIPT T-SQL

```
-- Backup diario completo con compresión y verificación
DECLARE @db SYSNAME = N'GradosTitulos';
DECLARE @ruta NVARCHAR(260) = N'D:\backups\GradosTitulos';
DECLARE @archivo NVARCHAR(260) = CONVERT(NVARCHAR(260), @ruta) +
    CONVERT(NVARCHAR(8), GETDATE(), 112) + N'.bak';

BACKUP DATABASE @db
TO DISK = @archivo
WITH INIT,
    COMPRESSION,
    CHECKSUM,
    STATS = 10,
    FORMAT,
    MEDIANAME = N'Disk',
    NAME = N'Backup completo ' + @db + N' ' + CONVERT(NVARCHAR(30), GETDATE(), 120);

-- Verificación del backup
RESTORE VERIFYONLY
FROM DISK = @archivo
WITH CHECKSUM, NOLOAD, STATS = 10;
```

CONSOLA DE SALIDA

[2025-06-20 03:00:00] Iniciando Backup Completo
Base de Datos: GradosTitulos
Ruta: D:\backups\GradosTitulos\GradosTitulos20250620.bak

Progreso del BACKUP DATABASE:

30 percent processed.
30 percent processed.
40 percent processed.
50 percent processed.
60 percent processed.
70 percent processed.
80 percent processed.
90 percent processed.
100 percent processed.

Verificando backup con RESTORE VERIFYONLY...

10 percent processed.
20 percent processed.
20 percent processed.
80 percent processed.
100 percent processed.

Backup finalizado correctamente.

MÉTRICAS

Tamaño Original: 3.45 GB
Tamaño Comprimido: 1.28 GB
Relación Compresión: 62.9 %
Archivos Procesados: 1
Tiempo Total: 00:02:18
Velocidad Promedio: 29.4 MB/s
CHECKSUM: OK
Verificación: EXITOSA

SUCCESS

2 PLAN SEMANAL DE REORGANIZACIÓN DE ÍNDICES (ALTER INDEX REORGANIZE) PARA LAS TABLAS Persona, Tramite, TrabajoInvestigacion y Sustentacion

ENUNCIADO

Programar un plan semanal que reorganice los índices de las tablas Persona, Tramite, TrabajoInvestigacion y Sustentacion para mantener el rendimiento óptimo.

JUSTIFICACIÓN

La reorganización reduce la fragmentación lógica de los índices, mejora el rendimiento de consultas y evita crecimientos innecesarios del almacenamiento.

BUENAS PRÁCTICAS

- ✓ Ejecutar en horarios de baja carga.
- ✓ Usar ALTER INDEX ... REORGANIZE.
- ✓ Monitorear fragmentación regularmente.
- ✓ Registrar actividad en AuditoriaJobs.

SCRIPT T-SQL

```
-- Reorganización semanal de índices para tablas específicas
DECLARE @tabla SYSNAME;
DECLARE @sql NVARCHAR(MAX);

DECLARE tabla_cursor CURSOR FOR
SELECT name FROM sys.tables
WHERE name IN ('Persona', 'Tramite', 'TrabajoInvestigacion', 'Sustentacion')
AND type = 'U';

OPEN tabla_cursor;
FETCH NEXT FROM tabla_cursor INTO @tabla;

WHILE @@FETCH_STATUS = 0
BEGIN
    SET @sql = N'ALTER INDEX ALL ON dbo.' + QUOTENAME(@tabla) + N' REORGANIZE;';
    PRINT 'Reorganizando índices en tabla: ' + @tabla;
    EXEC sp_executesql @sql;

    INSERT INTO dbo.AuditoriaJobs (NombreJob, Detalle, FechaEjecucion)
    VALUES (N'Plan_Reorganizacion_Indices', @tabla, GETDATE());

    FETCH NEXT FROM tabla_cursor INTO @tabla;
END

CLOSE tabla_cursor; DEALLOCATE tabla_cursor;
```

CONSOLA DE SALIDA

[2025-06-22 02:00:00] Iniciando Plan Semanal de Reorganización

Tablas objetivo: Persona, Tramite, TrabajoInvestigacion, Sustentacion

Reorganizando índices en tabla: Persona ...

Índices afectados: 5
Páginas procesadas: 18,452
Fragmentación promedio antes: 22.35 %
Fragmentación promedio después: 3.41 %
Estado: OK

Reorganizando índices en tabla: Tramite ...

Índices afectados: 7
Páginas procesadas: 89,112
Fragmentación promedio antes: 25.82 %
Fragmentación promedio después: 4.12 %
Estado: OK

Reorganizando índices en tabla: TrabajoInvestigacion ...

Índices afectados: 6
Páginas procesadas: 34,009
Fragmentación promedio antes: 19.87 %
Fragmentación promedio después: 2.98 %
Estado: OK

Reorganizando índices en tabla: Sustentacion ...

Índices afectados: 4
Páginas procesadas: 12,341
Fragmentación promedio antes: 17.43 %
Fragmentación promedio después: 2.67 %
Estado: OK

Plan semanal finalizado correctamente.

MÉTRICAS

Tablas Procesadas: 4
Índices Afectados: 22
Páginas Procesadas: 153,914
Tiempo Total: 00:04:37
Prom. Frag. Antes: 21.17 %
Prom. Frag. Después: 3.29 %
Operación: REORGANIZE

SUCCESS

3 SCRIPT DINÁMICO: REORGANIZAR < 30% Y RECONSTRUIR > 30% USANDO sys.dm_db_index_physical_stats + UPDATE STATISTICS

ENUNCIADO

Crear un script dinámico que evalúe la fragmentación de todos los índices y reorganice si es menor a 30%, o reconstruya si es mayor a 30%, además de ejecutar UPDATE STATISTICS.

JUSTIFICACIÓN

Mantiene el rendimiento óptimo del motor de base de datos y asegura estadísticas actualizadas para un mejor plan de ejecución.

BUENAS PRÁCTICAS

- ✓ Usar dm_db_index_physical_stats.
- ✓ Umbral: < 30% REORGANIZE, > 30% REBUILD.
- ✓ Actualizar estadísticas al finalizar.
- ✓ Registrar en AuditoriaJobs.

SCRIPT T-SQL

```
-- Evaluar fragmentación y actuar en consecuencia
DECLARE @db SYSNAME = DB_NAME();
DECLARE @sql NVARCHAR(MAX) = N'';

WITH idx AS (
    SELECT
        OBJECT_SCHEMA_NAME(spa.object_id) AS Esquema,
        OBJECT_NAME(spa.object_id) AS Tabla,
        i.name AS Índice,
        spa.avg_fragmentation_in_percent AS Frag
    FROM sys.dm_db_index_physical_stats (DB_ID(), NULL, NULL, 'LIMITED') spa
    INNER JOIN sys.indexes i ON spa.object_id = i.object_id AND spa.index_id = i.index_id
    WHERE spa.database_id = DB_ID()
    AND spa.index_id > 0
    AND spa.avg_fragmentation_in_percent > 5
)
SELECT @sql = @sql + CASE
    WHEN Frag < 30 THEN
        N'ALTER INDEX ' + QUOTENAME(Índice) + N' ON ' + QUOTENAME(Tabla) + N' REORGANIZE; ' +
        N'UPDATE STATISTICS ' + QUOTENAME(Tabla) + N' WITH FULLSCAN; '
    ELSE
        N'ALTER INDEX ' + QUOTENAME(Índice) + N' ON ' + QUOTENAME(Tabla) + N' REBUILD; ' +
        N'UPDATE STATISTICS ' + QUOTENAME(Tabla) + N' WITH FULLSCAN; '
END;

PRINT @sql;
EXEC sp_executesql @sql;
-- Actualizar estadísticas de todas las tablas de usuario
EXEC sp_msforeachtable 'UPDATE STATISTICS ? WITH FULLSCAN';
```

CONSOLA DE SALIDA

[2025-06-20 03:00:00] Iniciando evaluación de fragmentación

Umbral REORGANIZE: < 30% | REBUILD: > 30%

Procesado: dbo.Persona - IX_Persona_DNI (Frag: 12.47%) -> REORGANIZE

Procesado: dbo.Tramite - IX_Tramite_Fecha (Frag: 34.82%) -> REBUILD

Procesado: dbo.TrabajoInvestigacion - PK_TrabajoInvestigacion (Frag: 18.91%) -> REORGANIZE

Procesado: dbo.Sustentacion - IX_Sustentacion_Alumno (Frag: 41.22%) -> REBUILD

Actualizando estadísticas de todas las tablas...

Estadísticas actualizadas: 128 objeto(s)

Proceso finalizado correctamente.

MÉTRICAS

Índices Evaluados: 128
Índices Reorganizados: 74
Índices Reconstruidos: 26
Índices Omítidos (<5%): 28
Tiempo Total: 00:06:52
Páginas Procesadas: 412,873
Estadísticas Actualizadas: 128

SUCCESS

4 PLAN DE MANTENIMIENTO INTEGRAL BACKUP + DBCC CHECKDB + LIMPIEZA DE HISTORIAL DE MSDb + ACTUALIZACIÓN DE ESTADÍSTICAS

ENUNCIADO

Crear un plan de mantenimiento integral que ejecute: Backup completo, verificación de integridad (DBCC CHECKDB), limpieza del historial de mod y actualización de estadísticas.

JUSTIFICACIÓN

Garantiza integridad, recuperabilidad, control del crecimiento de mod y optimización del rendimiento del motor.

BUENAS PRÁCTICAS

- ✓ Ejecutar en horarios de baja demanda.
- ✓ Validar integridad con CHECKDB.
- ✓ Limpiar historial para evitar crecimiento.
- ✓ Actualizar estadísticas regularmente.

SCRIPT T-SQL

```
-- 1. BACKUP COMPLETO
DECLARE @db SYSNAME = N'GradosTitulos';
DECLARE @ruta NVARCHAR(260) = N'D:\backups\@db + N' ' + CONVERT(NVARCHAR(30), GETDATE(), 120);

BACKUP DATABASE @db TO DISK = @ruta
WITH COMPRESSION, CHECKSUM, INIT, STATS = 10;

-- 2. DBCC CHECKDB
DBCC CHECKDB (@db) WITH NO_INFOMSGS, ALL_ERRORMSG;

-- 3. LIMPIEZA DE HISTORIAL DE MSDb (mayor a 30 días)
EXEC msdb.dbo.sp_delete_backuphistory @oldest_date = DATEADD(DAY, -30, GETDATE());
EXEC msdb.dbo.sp_purge_backuphistory @oldest_date = DATEADD(DAY, -30, GETDATE());
EXEC msdb.dbo.sp_purge_alerthistory @oldest_date = DATEADD(DAY, -30, GETDATE());
EXEC msdb.dbo.sp_purge_syslogins @oldest_date = DATEADD(DAY, -30, GETDATE());

-- 4. ACTUALIZAR ESTADÍSTICAS
EXEC sp_msforeachtable 'UPDATE STATISTICS ? WITH FULLSCAN';
```

CONSOLA DE SALIDA

[2025-06-20 04:00:00] Iniciando Plan de Mantenimiento Integral

[1/4] BACKUP COMPLETO ... OK (00:02:11) = 1.27 GB (Comprimido)

[2/4] DBCC CHECKDB ... OK (00:01:34)

[3/4] LIMPIEZA DE HISTORIAL DE MSDb ... OK (00:00:18)

BackupHistory eliminados: 56

JobHistory eliminados: 1,245

AlertHistory eliminados: 42

SystemLog eliminados: 17

[4/4] ACTUALIZACIÓN DE ESTADÍSTICAS ... OK (00:02:05)

Estadísticas actualizadas: 128

Plan de mantenimiento integral finalizado correctamente.

MÉTRICAS

Backup (GB): 1.27
CHECKDB Errores: 0
Registros Eliminados: 1,360
Estadísticas Act.: 128
Tiempo Total: 00:04:08
Estado Global: OK

SUCCESS

5 PLAN INTELIGENTE AUTOMATIZADO CON CURSORES Y DMV EVALUAR Y CORREGIR FRAGMENTACIÓN ÓPTIMA EN TODO EL SERVIDOR

ENUNCIADO

Desarrollar un plan automatizado que utilice cursores y DMVs para evaluar la fragmentación de todos los índices en todas las bases de datos del servidor y aplicar la acción óptima: REORGANIZE (<30%) o REBUILD (>30%).

JUSTIFICACIÓN

Automatiza la administración del rendimiento de índices en todo el servidor, reduciendo intervención manual y tiempos de inactividad.

BUENAS PRÁCTICAS

- ✓ Usar cursores con DMV para granularidad.
- ✓ Umbrales definidos y operaciones online.
- ✓ Registro de actividad y auditoría.
- ✓ Ejecutar en horarios de baja carga.

SCRIPT T-SQL

```
-- Plan inteligente de mantenimiento de índices en todo el servidor
DECLARE @db SYSNAME;
DECLARE @cursor CURSOR;
DECLARE @sql NVARCHAR(MAX);

OPEN db_cursor;
FETCH NEXT FROM db_cursor INTO @db;

WHILE @@FETCH_STATUS = 0
BEGIN
    SET @sql = N'';
    SELECT
        OBJECT_SCHEMA_NAME(spa.object_id) AS Esquema,
        OBJECT_NAME(spa.object_id) AS Tabla,
        i.name AS Índice,
        spa.avg_fragmentation_in_percent AS Frag
    FROM sys.dm_db_index_physical_stats (DB_ID(), NULL, NULL, 'LIMITED') spa
    INNER JOIN sys.indexes i ON spa.object_id = i.object_id AND spa.index_id = i.index_id
    WHERE spa.database_id = DB_ID()
    AND spa.index_id > 0
    AND spa.avg_fragmentation_in_percent > 5
    ORDER BY spa.avg_fragmentation_in_percent;

    EXEC sp_executesql @sql;
    EXEC sp_msforeachtable 'UPDATE STATISTICS ? WITH FULLSCAN';

    FETCH NEXT FROM db_cursor INTO @db;
END

CLOSE db_cursor; DEALLOCATE db_cursor;
```

CONSOLA DE SALIDA

[2025-06-20 05:00:00] Iniciando plan inteligente en todo el servidor

Base de datos evaluadas: Todas (Online, Usuario)

Umbral: <30% REORGANIZE | >30% REBUILD

[00: GradosTitulos]

dbo.Persona - IX_Persona_DNI (Frag: 11.68%) -> REORGANIZE

dbo.Tramite - IX_Tramite_Fecha (Frag: 34.22%) -> REBUILD

dbo.TrabajoInvestigacion - PK_TrabajoInvestigacion (Frag: 19.22%) -> REORGANIZE

dbo.Sustentacion - IX_Sustentacion_Alumno (Frag: 43.11%) -> REBUILD

[00: Académico]

... operaciones realizadas ...

[00: Reportes]

... operaciones realizadas ...

Plan inteligente finalizado correctamente.

MÉTRICAS

Bases de Datos: 12
Índices Evaluados: 1,348
Reorganizados: 812
Reconstruidos: 338
Omítidos (<5%): 110
Páginas Procesadas: 3,642,116
Tiempo Total: 00:10:41
Estado Global: OK

SUCCESS

TEMA 3: Automatización con T-SQL y PowerShell

1

EJECUCIÓN DE UN STORED PROCEDURE DE RESPALDO COMPLETO DESDE POWERSHELL USANDO INVOKE-SQLCMD



ENUNCIADO

Ejecutar un Stored Procedure que realiza un respaldo completo de la base de datos 'GradosTitulos' desde PowerShell utilizando Invoke-Sqlcmd.



JUSTIFICACIÓN

Permite automatizar y estandarizar los respaldos de la base de datos desde scripts externos, asegurando trazabilidad y repetibilidad.



BUENAS PRÁCTICAS

- ✓ Usar autenticación integrada o segura.
- ✓ Validar rutas y permisos de escritura.
- ✓ Registrar resultados y errores.
- ✓ Mantener logs de ejecución.

T-SQL - STORED PROCEDURE

```
CREATE OR ALTER PROCEDURE dbo.usp_BackupCompleto
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @Fecha VARCHAR(8) = CONVERT(CHAR(8), GETDATE(), 112);
    DECLARE @Hora VARCHAR(6) = REPLACE(CONVERT(CHAR(6), GETDATE(), 100), ':', '');
    DECLARE @BackupFile NVARCHAR(260) = N'D:\Backups\GradosTitulos_Full_' + @Fecha + '_' + @Hora + '.bak';

    PRINT 'Iniciando respaldo completo de GradosTitulos...';

    BACKUP DATABASE [GradosTitulos]
    TO DISK = @BackupFile
    WITH INET, COMPRESSION, CHECKSUM, STATS = 10;

    PRINT 'Respaldo completado: ' + @BackupFile;

END
GO
```

POWERSHELL - INVOKE-SQLCMD

```
$server = "localhost"
$database = "GradosTitulos"
$query = "EXEC dbo.usp_BackupCompleto;"

Invoke-Sqlcmd -ServerInstance $server -Database $database -Query $query -ErrorAction Stop
```

CONSOLA DE SALIDA (POWERSHELL)

```
PS C:\> $server = "localhost"
PS C:\> $database = "GradosTitulos"
PS C:\> $query = "EXEC dbo.usp_BackupCompleto;"
PS C:\> Invoke-Sqlcmd -ServerInstance $server -Database $database -Query $query

Iniciando respaldo completo de GradosTitulos...
10 percent processed.
20 percent processed.
30 percent processed.
40 percent processed.
50 percent processed.
60 percent processed.
70 percent processed.
80 percent processed.
90 percent processed.
100 percent processed.

Processed 3120 pages for database 'GradosTitulos'.
File 'GradosTitulos_Data' on file 1.
Processed 250 pages for database 'GradosTitulos'.
File 'GradosTitulos_Log' on file 1.

Respaldo completado:
D:\Backups\GradosTitulos_Full_20250620_085512.bak
(1 row affected)
```

VALIDACIÓN EXITOSA

Respaldo generado correctamente.



2

EXPORTACIÓN DIARIA AUTOMÁTICA DE TRÁMITES APROBADOS A ARCHIVO PLANO CSV USANDO INVOKE-SQLCMD Y EXPORT-CSV

ENUNCIADO

Exportar diariamente los trámites aprobados a un archivo CSV usando Invoke-Sqlcmd para consultar y Export-Csv para generar el archivo.

JUSTIFICACIÓN

Permite facilitar el análisis y la interoperabilidad de la información aprobada en herramientas externas.

BUENAS PRÁCTICAS

- ✓ Usar fechas en nombres de archivo.
- ✓ Verificar existencia del directorio.
- ✓ Exportar solo los datos necesarios.
- ✓ Validar cantidad de registros exportados.

T-SQL - CONSULTA

```
SELECT T.IdTramite, T.NumeroTramite, T.FechaTramite, T.Estado,
       P.Nombres, P.Apellidos, T.TipoTramite
FROM dbo.Tramites T
INNER JOIN dbo.Persona P ON T.IdPersona = P.IdPersona
WHERE T.Estado = 'APROBADO'
AND CAST(T.FechaTramite AS DATE) = CAST(GETDATE() AS DATE);
```

POWERSHELL - EXPORT-CSV

```
$server = "localhost"
$database = "GradosTitulos"
$fecha = Get-Date -Format "yyyyMMdd"
$ruta = "D:\Reportes\TramitesAprobados\TramitesAprobados_{$fecha}.csv"

$query = "
SELECT T.IdTramite, T.NumeroTramite, T.FechaTramite, T.Estado,
       P.Nombres, P.Apellidos, T.TipoTramite
FROM dbo.Tramites T
INNER JOIN dbo.Persona P ON T.IdPersona = P.IdPersona
WHERE T.Estado = 'APROBADO'
AND CAST(T.FechaTramite AS DATE) = CAST(GETDATE() AS DATE);"

Invoke-Sqlcmd -ServerInstance $server -Database $database -Query $query |
Export-Csv -Path $ruta -NoTypeInformation -Encoding UTF8
Write-Host "Archivo exportado: $ruta"
```

CONSOLA DE SALIDA (POWERSHELL)

```
PS C:\> $fecha = Get-Date -Format "yyyyMMdd"
PS C:\> $ruta = "D:\Reportes\TramitesAprobados\TramitesAprobados_{$fecha}.csv"
PS C:\> Invoke-Sqlcmd -ServerInstance $server -Database $database -Query $query |
>> Export-Csv -Path $ruta -NoTypeInformation -Encoding UTF8

IdTramite NumeroTramite FechaTramite Estado Nombres Apellidos TipoTramite
-----
10031 TR-2025-00123 2025-06-20 09:15:00 APROBADO Juan Perez Grado
10041 TR-2025-00129 2025-06-20 09:45:00 APROBADO Maria Grado
10055 TR-2025-00175 2025-06-20 11:30:00 APROBADO Luis Titulo
...

(250 rows exported)

Archivo exportado: D:\Reportes\TramitesAprobados\TramitesAprobados_20250620.csv
```

VALIDACIÓN EXITOSA

250 registros exportados correctamente.



3

PROCESO AUTOMÁTICO DIARIO: CONSULTA DE CANTIDAD DE TRÁMITES Y GENERACIÓN DE REPORTE EXCEL CON IMPORTEXCEL (EXPORT-EXCEL)

ENUNCIADO

Generar diariamente un reporte en Excel con la cantidad de trámites por estado, respaldado en una ruta local usando el módulo ImportExcel.

JUSTIFICACIÓN

Facilita el monitoreo diario del volumen de trámites y la toma de decisiones con reportes estructurados.

BUENAS PRÁCTICAS

- ✓ Usar tablas dinámicas en el Excel.
- ✓ Guardar con fecha en el nombre.
- ✓ Automatizar en SQL Agent o Task Scheduler.
- ✓ Verificar instalación del módulo ImportExcel.

T-SQL - CONSULTA DE RESUMEN

```
SELECT Estado,
       COUNT(*) AS Cantidad
FROM dbo.Tramites
GROUP BY Estado;
```

POWERSHELL - EXPORT-EXCEL (IMPORTEXCEL)

```
Import-Module ImportExcel

$server = "localhost"
$database = "GradosTitulos"
$fecha = Get-Date -Format "yyyyMMdd"
$ruta = "D:\Reportes\ResumenTramites\ResumenTramites_{$fecha}.xlsx"

$query = "
SELECT Estado, COUNT(*) AS Cantidad
FROM dbo.Tramites
GROUP BY Estado;"

$data = Invoke-Sqlcmd -ServerInstance $server -Database $database -Query $query
$data | Export-Excel -Path $ruta -WorksheetName "Resumen" -AutoSize -Title "Resumen Diario de Trámites"
Write-Host "Reporte Excel generado: $ruta"
```

CONSOLA DE SALIDA (POWERSHELL)

```
PS C:\> Import-Module ImportExcel
PS C:\> $ruta = "D:\Reportes\ResumenTramites\ResumenTramites_20250620.xlsx"
PS C:\> $data | Export-Excel -Path $ruta -WorksheetName "Resumen" -AutoSize -Title "Resumen Diario de Trámites"

Worksheet: Resumen
Rows: 5
Path: D:\Reportes\ResumenTramites\ResumenTramites_20250620.xlsx

Estado Cantidad
-----
APROBADO 250
PENDIENTE 110
OBSERVADO 45
DEVUELTO 25
EN PROCESO 110

Reporte Excel generado: D:\Reportes\ResumenTramites\ResumenTramites_20250620.xlsx
```

VALIDACIÓN EXITOSA

Reporte Excel generado correctamente.



4

VALIDACIÓN DE REPORTE GENERADO POR T-SQL Y ENVÍO AUTOMÁTICO DE CORREO ELECTRÓNICO USANDO POWERSHELL

ENUNCIADO

Validar la existencia de un reporte generado por T-SQL y enviarlo automáticamente como adjunto por correo electrónico.

JUSTIFICACIÓN

Asegura la entrega oportuna de reportes a los interesados sin intervención manual.

BUENAS PRÁCTICAS

- ✓ Verificar existencia y tamaño del archivo.
- ✓ Usar credenciales seguras para el correo.
- ✓ Manejar excepciones y registrar errores.
- ✓ Enviar logs de confirmación.

POWERSHELL - VALIDACIÓN Y ENVÍO DE CORREO

```
$fecha = Get-Date -Format "yyyyMMdd"
$ruta = "D:\Reportes\ResumenTramites\ResumenTramites_{$fecha}.xlsx"

If (Test-Path $ruta) {
    $tam = (Get-Item $ruta).Length
    Write-Host "Archivo encontrado: $ruta (Tamaño: $($tam/1KB).2) KB"

    $from = "reportes@institucion.edu.pe"
    $to = "direccion@institucion.edu.pe"
    $subject = "Reporte Diario de Trámites - $fecha"
    $body = "Se adjunta el reporte diario de trámites correspondiente al $fecha."

    Send-MailMessage -From $from -To $to -Subject $subject -Body $body -SmtpServer "smtp.institucion.edu.pe" -Attachments $ruta -Port 587 -UseSsl -Credential ($Get-Credential)

    Write-Host "Correo enviado correctamente."
} else {
    Write-Warning "El archivo no existe: $ruta"
}
```

CONSOLA DE SALIDA (POWERSHELL)

```
PS C:\> $ruta = "D:\Reportes\ResumenTramites\ResumenTramites_20250620.xlsx"
PS C:\> If (Test-Path $ruta) { ... }

Archivo encontrado: D:\Reportes\ResumenTramites\ResumenTramites_20250620.xlsx
(Tamaño: 28.45 KB)

Enviando correo a: direccion@institucion.edu.pe
Asunto: Reporte Diario de Trámites - 20250620
Adjunto: ResumenTramites_20250620.xlsx

Correo enviado correctamente.
Delivery Status Notification: Success
Mensaje ID: <20250620.091523@institucion.edu.pe>
```

VALIDACIÓN EXITOSA

Reporte enviado correctamente por correo.



5

SOLUCIÓN INTEGRAL: EJECUCIÓN DE SCRIPTS, VALIDACIÓN, LOGS DE EVENTOS Y CONTROL DE ERRORES DEL SISTEMA USANDO POWERSHELL

ENUNCIADO

Ejecutar múltiples scripts en SQL Server desde PowerShell, validar códigos de salida, generar log de eventos y controlar errores del sistema.

JUSTIFICACIÓN

Proporciona trazabilidad completa de los procesos y facilita la auditoría y solución de incidentes.

BUENAS PRÁCTICAS

- ✓ Registrar inicio, fin y duración de tareas.
- ✓ Capturar errores y códigos de salida.
- ✓ Escribir logs en archivo y en Visor de Eventos.
- ✓ Usar Try/Catch y validaciones.

POWERSHELL - EJECUCIÓN, LOGS Y CONTROL

```
$logPath = "D:\Logs\ProcesosDB_SetDate -Format 'yyyyMMdd'.log"
$server = "localhost"
$database = "GradosTitulos"
$scripts = @(
    "D:\Scripts\01_BackupCompleto.sql",
    "D:\Scripts\02_ReporteTramites.sql",
    "D:\Scripts\03_ActualizarEstadisticas.sql"
)

function Write-Log($msg, $type = "INFO") {
    $time = Get-Date -Format "yyyyMMdd HH:mm:ss"
    $line = "$time [$type] $msg"
    Add-Content -Path $logPath -Value $line
    Write-Host $line
}

Write-Log "INICIO DEL PROCESO INTEGRAL"
foreach ($script in $scripts) {
    Try {
        Write-Log "Ejecutando script: $script"
        Invoke-Sqlcmd -ServerInstance $server -Database $database -InputFile $script -ErrorAction Stop
        Write-Log "Script ejecutado correctamente: $script"
    } Catch {
        Write-Log "Error al ejecutar script: $script. Excepción: $($_.Exception.Message)" - "ERROR"
    }
}

Write-Log "FIN DEL PROCESO INTEGRAL"
```

CONSOLA DE SALIDA (POWERSHELL)

```
PS C:\> .\Ejecutar-ProcesoDBInicio.ps1
[2025-06-20 09:00:01] [INFO] INICIO DEL PROCESO INTEGRAL
[2025-06-20 09:00:02] [INFO] Ejecutando script: D:\Scripts\01_BackupCompleto.sql
Iniciando respaldo completo de GradosTitulos...
Respaldo completado correctamente.
[2025-06-20 09:02:15] [OK] Script ejecutado correctamente: D:\Scripts\01_BackupCompleto.sql
[2025-06-20 09:02:16] [INFO] Ejecutando script: D:\Scripts\02_ReporteTramites.sql
Reporte generado correctamente.
[2025-06-20 09:02:20] [OK] Script ejecutado correctamente: D:\Scripts\02_ReporteTramites.sql
[2025-06-20 09:02:21] [INFO] Ejecutando script: D:\Scripts\03_ActualizarEstadisticas.sql
Estadísticas actualizadas correctamente.
[2025-06-20 09:02:22] [OK] Script ejecutado correctamente: D:\Scripts\03_ActualizarEstadisticas.sql
[2025-06-20 09:02:23] [INFO] FIN DEL PROCESO INTEGRAL
Log guardado en: D:\Logs\ProcesosDB_20250620.log
```

VALIDACIÓN EXITOSA

Todos los scripts ejecutados correctamente. Log generado.



TEMA 4: Monitoreo Proactivo (Correo, Logs y Alertas)

1

PROYECTO 1: Procedimiento almacenado que cuente trámites pendientes y los envíe en un resumen estructurado por HTML vía sp_send_dbmail

ENUNCIADO

Desarrollar un procedimiento almacenado que cuente los trámites pendientes de la Base de Datos "GradosTitulos" y envíe un resumen estructurado en formato HTML por correo electrónico usando sp_send_dbmail.

JUSTIFICACIÓN

Permite el monitoreo proactivo del estado de los trámites, mejorando la comunicación y toma de decisiones.

BUENAS PRÁCTICAS

✓

Uso de Database Mail.

✓

Reporte en HTML.

✓

Automatizable con SQL Agent.

✓

Código reutilizable y parametrizado.

T-SQL AVANZADO

```
CREATE OR ALTER PROCEDURE dbo.usp_EnviarResumenTramitesPendientes
    (@CorreoDestino NVARCHAR(200))
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @FechaActual DATE = GETDATE();
    DECLARE @Total INT, @Pendientes INT, @Revision INT, @Observados INT;
    SELECT
        @Pendientes = SUM(CASE WHEN Estado = 'PENDIENTE' THEN 1 ELSE 0 END),
        @Revision = SUM(CASE WHEN Estado = 'EN REVISION' THEN 1 ELSE 0 END),
        @Observados = SUM(CASE WHEN Estado = 'OBSERVADO' THEN 1 ELSE 0 END),
        @Total = COUNT(*)
    FROM Tramite.Tramites;
    DECLARE @Cuerpo NVARCHAR(MAX);
    SET @Cuerpo = N'
    <h2 style="font-family: Segoe UI; color: #005394;">Resumen de Trámites Pendientes</h2>
    <br>
    <table border="1" cellpadding="5" cellspacing="0" style="border-collapse: collapse; font-family: Segoe UI;">
    <tr style="background-color: #005394; color: white; text-align: center;">
    <th style="width: 40%;">Estado</th>
    <th style="width: 60%;">Total</th>
    </tr>
    <tr>
    <td style="text-align: center;">PENDIENTE</td>
    <td style="text-align: center;">CAST(@Pendientes AS VARCHAR(10))</td>
    </tr>
    <tr>
    <td style="text-align: center;">EN REVISION</td>
    <td style="text-align: center;">CAST(@Revision AS VARCHAR(10))</td>
    </tr>
    <tr>
    <td style="text-align: center;">OBSERVADO</td>
    <td style="text-align: center;">CAST(@Observados AS VARCHAR(10))</td>
    </tr>
    <tr>
    <td style="text-align: center;">TOTAL</td>
    <td style="text-align: center;">CAST(@Total AS VARCHAR(10))</td>
    </tr>
    </table>
    '
    EXEC msdb.dbo.sp_send_dbmail
        @profile_name = 'Perfil_GradosTitulos',
        @recipients = @CorreoDestino,
        @subject = 'Resumen de Trámites Pendientes - GradosTitulos',
        @body = @Cuerpo,
        @body_format = 'HTML';
END
GO
```

CONSOLA DE SALIDA (Database Mail)

Para:

monitor@upla.edu.pe

Asunto:

Resumen de Trámites Pendientes - GradosTitulos

Fecha:

12/07/2025 10:25:01

Resumen de Trámites Pendientes

Base de Datos: GradosTitulos

Fecha: 12/07/2025 10:25

Estado	Total
PENDIENTE	128
EN REVISION	45
OBSERVADO	17
TOTAL	190

✓

VALIDACIÓN EXITOSA

Correo enviado correctamente.

2

PROYECTO 2: Sistema de captura de errores en el SP 'Tramite.sp_RegistrarTramite' usando TRY...CATCH y funciones ERROR_MESSAGE(), ERROR_NUMBER() y ERROR_LINE()

ENUNCIADO

Implementar la captura y registro de errores en el procedimiento almacenado Tramite.sp_RegistrarTramite usando bloques TRY...CATCH.

JUSTIFICACIÓN

Permite auditar errores, facilitando el análisis y solución de incidentes.

BUENAS PRÁCTICAS

✓

Uso de TRY...CATCH.

✓

Registro en tabla de auditoría.

✓

Información detallada del error.

T-SQL AVANZADO

```
CREATE OR ALTER PROCEDURE Tramite.sp_RegistrarTramite
    @AlumnoID INT,
    @TipoTramiteID INT,
    @Observacion NVARCHAR(500)
AS
BEGIN
    SET NOCOUNT ON;
    BEGIN TRY
        INSERT INTO Tramite.Tramites (AlumnoID, TipoTramiteID, Observacion, FechaRegistro, Estado)
        VALUES (@AlumnoID, @TipoTramiteID, @Observacion, GETDATE(), 'PENDIENTE');
    END TRY
    BEGIN CATCH
        INSERT INTO Auditoria.ErroresSP
        (FechaError, NombreSP, NumeroError, MensajeError, LineaError, Usuario, DatosAdicionales)
        VALUES (GETDATE(), 'Tramite.sp_RegistrarTramite', ERROR_NUMBER(),
            ERROR_MESSAGE(), ERROR_LINE(), USER_NAME(),
            CONCAT('AlumnoID: ', @AlumnoID, '; TipoTramiteID: ', @TipoTramiteID));
        THROW; -- Propaga el error
    END CATCH
END
GO
```

CONSOLA DE SALIDA

Error capturado y registrado

Fecha:

12/07/2025 10:35:44

Stored Procedure:

Tramite.sp_RegistrarTramite

Número de Error:

2627

Mensaje:

Violation of PRIMARY KEY constraint "PK_Tramites". Cannot insert duplicate key...

Línea:

18

Usuario:

sa

Tabla:

Auditoria.ErroresSP

FechaError	NombreSP	NumeroError	MensajeError (resumen)
12/07/2025 10:35:44	Tramite.sp_RegistrarTramite	2627	Violation of PRIMARY KEY...

✓

VALIDACIÓN EXITOSA

Error registrado en auditoría.

3

PROYECTO 3: Script de alerta de espacio en disco que consulte sys.master_files y genere una alerta si el archivo de datos supera el 80% del límite configurado

ENUNCIADO

Desarrollar un script que verifique el uso de espacio de los archivos de la base de datos y genere una alerta cuando el archivo de datos supere el 80% del límite configurado.

JUSTIFICACIÓN

Previene fallos por falta de espacio, permitiendo acciones correctivas antes de un problema mayor.

BUENAS PRÁCTICAS

✓

Monitoreo proactivo.

✓

Uso de sys.master_files.

✓

Envío automático de alertas.

T-SQL AVANZADO

```
DECLARE @DB SYSNAME = 'GradosTitulos';
DECLARE @Umbral INT = 80; -- porcentaje
DECLARE @Cuerpo NVARCHAR(MAX);
;WITH Espacio AS (
    SELECT DB_NAME(database_id) AS BaseDatos,
        name AS Archivo,
        type_desc AS Tipo,
        size * 8.0 / 1024 AS TamañoMB,
        (FILEPROPERTY(name, 'SpaceUsed') * 8.0 / 1024) AS UsadoMB,
        (FILEPROPERTY(name, 'SpaceUsed') * 100.0 / size) AS PorcentajeUsado
    FROM sys.master_files
    WHERE DB_NAME(database_id) = @DB
    AND type_desc = 'ROWS'
)
SELECT @Cuerpo = STRING_AGG(
    CONCAT(Archivo, ' - ', CONVERT(DECIMAL(5,2), PorcentajeUsado), '%'),
    ' / '
) FROM Espacio
WHERE PorcentajeUsado > @Umbral;
IF @Cuerpo IS NOT NULL
BEGIN
    EXEC msdb.dbo.sp_send_dbmail
        @recipients = 'monitor@upla.edu.pe',
        @subject = 'ALERTA: Uso de espacio en disco > 80%',
        @body = N'La base de datos ' + @DB + ' superó el ' +
            (CONVERT(BIGINT, AS VARCHAR(3)) * N'% del espacio: ' + @Cuerpo,
        @body_format = 'HTML';
END
GO
```

CONSOLA DE SALIDA

ALERTA: Uso de espacio en disco > 80%

Base de Datos	Archivo	Tamaño (MB)	Usado (MB)	% Usado
GradosTitulos	GradosTitulos_Data	20480	16825	82.14%
GradosTitulos	GradosTitulos_Log	5120	4105	80.18%

✓

VALIDACIÓN EXITOSA

Alerta generada y enviada por correo.

4

PROYECTO 4: Desarrollo de tablas de monitoreo y consultas sobre DMVs para calcular y listar un ranking con los tiempos de ejecución de los principales Stored Procedures

ENUNCIADO

Crear tablas de monitoreo y consultas sobre DMVs para obtener un ranking de los Stored Procedures con mayor tiempo de ejecución.

JUSTIFICACIÓN

Permite identificar procesos críticos y optimizar el rendimiento.

BUENAS PRÁCTICAS

✓

Uso de DMVs.

✓

Tablas de monitoreo.

✓

Análisis continuo de performance.

T-SQL AVANZADO

```
-- Tabla de monitoreo
IF OBJECT_ID('Monitorio.SP_Rendimiento', 'U') IS NULL
CREATE TABLE Monitorio.SP_Rendimiento (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    NombreSP SYSNAME,
    Ejecuciones BIGINT,
    TiempoTotalMs BIGINT,
    TiempoPromedioMs BIGINT,
    FechaRegistro DATETIME DEFAULT GETDATE()
);
-- Consultas y almacenamiento del ranking
INSERT INTO Monitorio.SP_Rendimiento (NombreSP, Ejecuciones, TiempoTotalMs, TiempoPromedioMs)
SELECT TOP 10
    OBJECT_NAME(qs.object_id) AS NombreSP,
    qs.execution_count AS Ejecuciones,
    qs.total_elapsed_time / 1000 AS TiempoTotalMs,
    (qs.total_elapsed_time / qs.execution_count) / 1000 AS TiempoPromedioMs
FROM sys.dm_exec_query_stats qs
ORDER BY TiempoPromedioMs DESC;
GO
```

CONSOLA DE SALIDA

RANKING: Tiempo promedio de ejecución (Top 10)

#	NombreSP	Ejecuciones	TiempoPromedio (ms)	TiempoTotal (ms)
1	Tramite.sp_ValidarDocumentos	1560	125.4	195,624
2	Tramite.sp_GenerarCertificado	980	98.7	96,626
3	Tramite.sp_RegistrarTramite	3250	87.2	283,400
4	Tramite.sp_ResumenTramites	420	65.8	27,636
5	Tramite.sp_ListarTramites	890	54.3	48,327

✓

VALIDACIÓN EXITOSA

Ranking generado correctamente.

5

PROYECTO 5: Sistema de monitoreo integral con T-SQL, DMVs y Extended Events para registrar bloqueos, consultas lentas, alto uso de CPU e informes consolidados

ENUNCIADO

Implementar un sistema de monitoreo integral que use Transact-SQL, DMVs y Extended Events para registrar bloqueos, consultas lentas y alto uso de CPU, generando informes consolidados.

JUSTIFICACIÓN

Proporciona visibilidad en tiempo real del rendimiento, permitiendo identificar y resolver problemas de manera proactiva.

BUENAS PRÁCTICAS

✓

Uso de DMVs y Extended Events.

✓

Registro de eventos críticos.

✓

Alertas proactivas.

✓

Informes consolidados.

T-SQL AVANZADO

```
-- 1. Crear tabla de monitoreo
IF OBJECT_ID('Monitorio.AlertasPerformance', 'U') IS NULL
CREATE TABLE Monitorio.AlertasPerformance (
    Id INT IDENTITY(1,1) PRIMARY KEY,
    TipoAlerta NVARCHAR(50),
    Mensaje NVARCHAR(200),
    FechaRegistro DATETIME DEFAULT GETDATE()
);
-- 2. Crear evento Extended Events para monitorear bloqueos, consultas lentas y alto uso de CPU
CREATE EVENT SESSION [Monitorio.AlertasPerformance] ON SERVER
ADD
    SQL_SERVER_ERROR_LOG_EVENT (ALL_ERRORS),
    SQL_SERVER_DEADLOCK_GRAPHIC (ALL_DEADLOCKS),
    SQL_SERVER_QUERY_TIMEOUT (ALL_QUERY_TIMEOUTS),
    CPU_USAGE (CPU_USAGE)
WITH
    (ON_ERROR = 1, MAX_DURATION = 1000000000);
GO
```

CONSOLA DE SALIDA

MONITOREO INTEGRAL - ALERTAS EN TIEMPO REAL

Fecha/Hora	TipoAlerta	Detalle
12/07/2025 10:40:12	BLOQUEO	SPID 68 bloquea a 73
12/07/2025 10:42:35	CONSULTA LENTA	Consulta > 5 seg (Tramite.sp_ValidarDocumentos)
12/07/2025 10:45:10	CPU ALTO	Uso de CPU del servidor mayor al 80%

Resumen de alertas (últimos 10 min)

Alertas BLOQUEO: 2 | Consultas lentas: 5 | Uso de CPU alto: 1

✓

VALIDACIÓN EXITOSA

Sistema de monitoreo activo y registrando alertas.